

Reverse Engineering Lab

Cheat Sheet

x64 assembly · Windows internals
Microsoft x64 calling convention

01 Data Types & Sizes

Sizes

TERM	BYTES	BITS
byte	1	8
word	2	16
dword	4	32
qword	8	64

Type sizes are for 64-bit Windows (LLP64): long stays 4 bytes, pointers are 8.

Common data types

TYPE	BYTES	ASM SIZE
char / bool	1	byte
short	2	word
int	4	dword
long	4	dword
long long	8	qword

TYPE	BYTES	ASM SIZE
float	4	dword
double	8	qword
pointer*	8	qword
wchar_t / char16_t	2	word
char32_t	4	dword

* also size_t, ptrdiff_t, intptr_t

02 Registers in x64 Assembly

General-purpose registers (can be changed)

64-BIT	32-BIT	16-BIT	8-BIT LOW	8-BIT HIGH	TRADITIONAL ROLE
RAX	EAX	AX	AL	AH	Accumulator, return value
RBX	EBX	BX	BL	BH	Base
RCX	ECX	CX	CL	CH	Counter, loop/string ops
RDX	EDX	DX	DL	DH	Data, I/O port, high half of mul/div
RSI	ESI	SI	SIL	—	Source index for string ops
RDI	EDI	DI	DIL	—	Destination index
RBP	EBP	BP	BPL	—	Base / frame pointer
RSP	ESP	SP	SPL	—	Stack pointer
R8	R8D	R8W	R8B	—	x64 only
R9	R9D	R9W	R9B	—	x64 only
R10–R15	R10D–R15D	R10W–R15W	R10B–R15B	—	x64 only

Special registers (cannot be changed directly)

64-BIT	32-BIT	16-BIT	ROLE
RIP	EIP	IP	Instruction pointer — address of the next instruction
RFLAGS	EFLAGS	FLAGS	Status flags set by arithmetic and logic

RFLAGS

BIT	FLAG	NAME	SET WHEN
0	CF	Carry	Unsigned overflow. A carry out of, or borrow into, the most significant bit
2	PF	Parity	The low 8 bits of the result contain an even number of set bits
4	AF	Auxiliary carry	Carry out of bit 3. BCD arithmetic only, no jump reads it
6	ZF	Zero	The result was zero
7	SF	Sign	The most significant bit of the result was set (negative, if signed)
11	OF	Overflow	Signed overflow. The result won't fit in the destination as a signed value

03 Common Assembly Instructions & Format

Instruction format

Destination always comes first

```
mnemonic destination, source
mov      rax, rcx      ; rax = rcx

mnemonic destination
inc      rax           ; rax = rax + 1

mnemonic
ret                      ; return to caller
```

Operands must be the same size

...unless the instruction is designed to handle mismatched sizes:

```
mov rax, ecx      ; ✗ 64-bit ≠ 32-bit
mov eax, ecx      ; ✓ 32-bit = 32-bit

movsxd rax, ecx   ; signed: fills rax's upper
                  ; 32 bits with ecx's sign bit
```

Square brackets mean dereference

Exactly like using a pointer in C. Given `rbx = 0x7ff000`, and `0x2a` is stored at `0x7ff000`:

```
mov rax, rbx      ; rax = rbx → 0x7ff000 (address)
mov rax, [rbx]     ; rax = *rbx → 0x2a (contents)
mov [rbx], rax     ; *rbx = rax
mov rax, [rbx+8]   ; reads from 0x7ff008
```

Important: you cannot dereference both the destination and the source in the same instruction.

```
mov [rbx], [rax] ; ✗ assembler error
```

Data movement

Copy values between registers, memory, and immediate constants. The source is left unchanged, and at most one operand may be memory.

INSTRUCTION	EFFECT	NOTES
<code>mov d, s</code>	<code>d = s</code>	At most one operand may be memory; no flags affected
<code>movabs r64, imm64</code>	<code>r64 = imm64</code>	The only form taking a full 64-bit immediate; plain <code>mov</code> sign-extends <code>imm32</code>
<code>movzx d, s</code>	<code>d = s, zero-extended</code>	For unsigned data; <code>s</code> is 8 or 16 bits
<code>movsx d, s</code>	<code>d = s, sign-extended</code>	For signed data; <code>0xFF</code> becomes <code>-1</code> , not <code>255</code>
<code>movsxd r64, r32</code>	<code>32 → 64 sign extension</code>	No <code>movzxd</code> exists — <code>mov eax, ecx</code> already zero-extends
<code>lea d, [expr]</code>	<code>d = address of expr</code>	Bracket syntax but no memory access; doesn't touch flags
<code>push s</code>	<code>rsp -= 8; [rsp] = s</code>	Always 8 bytes on x64, 4 on x86; immediates allowed
<code>pop d</code>	<code>d = [rsp]; rsp += 8</code>	—

Arithmetic

Integer arithmetic that updates the flags in RFLAGS. The result goes to the destination operand, except for `mul` and `div`, which use `RDX:RAX` implicitly.

INSTRUCTION	EFFECT	NOTES
<code>add d, s</code>	$d = d + s$	Sets CF, OF, SF, ZF
<code>sub d, s</code>	$d = d - s$	Sets CF, OF, SF, ZF
<code>inc d</code>	$d = d + 1$	Does not affect CF
<code>dec d</code>	$d = d - 1$	Does not affect CF
<code>neg d</code>	$d = -d$	Two's complement negate
<code>mul s</code>	$RDX:RAX = RAX * s$	Unsigned, one operand, implicit RAX/RDX
<code>imul d, s</code>	$d = d * s$	Signed, two-operand, no RDX
<code>div s</code>	$RAX = RDX:RAX / s$ $RDX = \text{remainder}$	Unsigned; faults if quotient overflows
<code>cmp d, s</code>	Computes $d - s$, discards result	Sets flags only

Watch RDX when dividing. `div` and `idiv` use RDX as the high half of the dividend, not scratch. Zero it (`xor edx, edx`) or sign-extend into it (`cqo`) before dividing, or you'll get arithmetic bugs.

Bitwise and shifts

Bitwise instructions apply the operation to each bit position independently. Shift and rotate instructions move the bits of an operand left or right by a given count.

INSTRUCTION	EFFECT	NOTES
<code>and d, s</code>	$d = d \& s$	Clears CF and OF
<code>or d, s</code>	$d = d s$	—
<code>xor d, s</code>	$d = d \wedge s$	<code>xor eax, eax</code> is the idiomatic zero
<code>not d</code>	$d = \sim d$	Does not affect flags
<code>test d, s</code>	Computes $d \& s$, discards result	Sets flags only
<code>shl / sal d, cnt</code>	Left shift, fill with 0	Same instruction
<code>shr d, cnt</code>	Logical right shift, fill with 0	Unsigned divide by 2^n
<code>sar d, cnt</code>	Arithmetic right shift, fill with sign	Signed divide by 2^n
<code>rol / ror d, cnt</code>	Rotate left / right	—

Control flow

Redirect execution by changing RIP instead of producing a value. Conditional jumps read the flags set by a previous instruction — usually `cmp` or `test` — rather than comparing anything themselves. `call` and `ret` also use the stack, pushing and popping the return address.

INSTRUCTION	EFFECT	NOTES
<code>jmp target</code>	Unconditional jump	Direct, register, or memory operand
<code>je / jz</code>	Jump if ZF=1	Equal / zero
<code>jne / jnz</code>	Jump if ZF=0	Not equal / not zero
<code>jg / jge / jl / jle</code>	Signed comparison	Uses SF, OF, ZF
<code>ja / jae / jb / jbe</code>	Unsigned comparison	Uses CF, ZF
<code>js / jns</code>	Jump if sign set / clear	—
<code>jc / jnc</code>	Jump if carry set / clear	—
<code>jo / jno</code>	Jump on signed overflow	—
<code>jp / jnp</code>	Jump on parity	Used in float comparison (NaN)
<code>jrcxz</code>	Jump if RCX is zero	No flags consulted
<code>call target</code>	Push return address, jump	—
<code>ret / ret n</code>	Pop return address, jump	<code>ret n</code> also pops <code>n</code> bytes of args
<code>loop target</code>	--RCX; jump if nonzero	Slow; avoid
<code>syscall</code>	Fast system call entry	x64; clobbers RCX and R11
<code>nop</code>	No operation	Alignment padding

04 The Stack & Functions

Working with the stack

- Stores variables local to functions, return addresses, and acts as scratch space
- Grows **downward** (high → low addresses); operates **LIFO**
- Big objects (strings, structs) are passed by pointer, not copied

Functions — Microsoft x64 calling convention

Defines how arguments are passed, where return values appear, and which registers survive a call.

Passing arguments

The first four arguments go in registers. Everything after that goes on the stack.

ARGUMENT	INTEGER / POINTER
1st	rcx
2nd	rdx
3rd	r8
4th	r9
5th+	[rsp+20h], [rsp+28h], ...

The slot index is positional, not per-type.

```
mov rcx, 1      ; arg 1
mov rdx, 2      ; arg 2
call TwoArgFunction
```

Stack requirements

Two rules the caller must satisfy before **every** call:

Shadow space — allocate 32 bytes (0x20) above the return address for the callee to spill its four register arguments. Required even when the function takes no arguments.

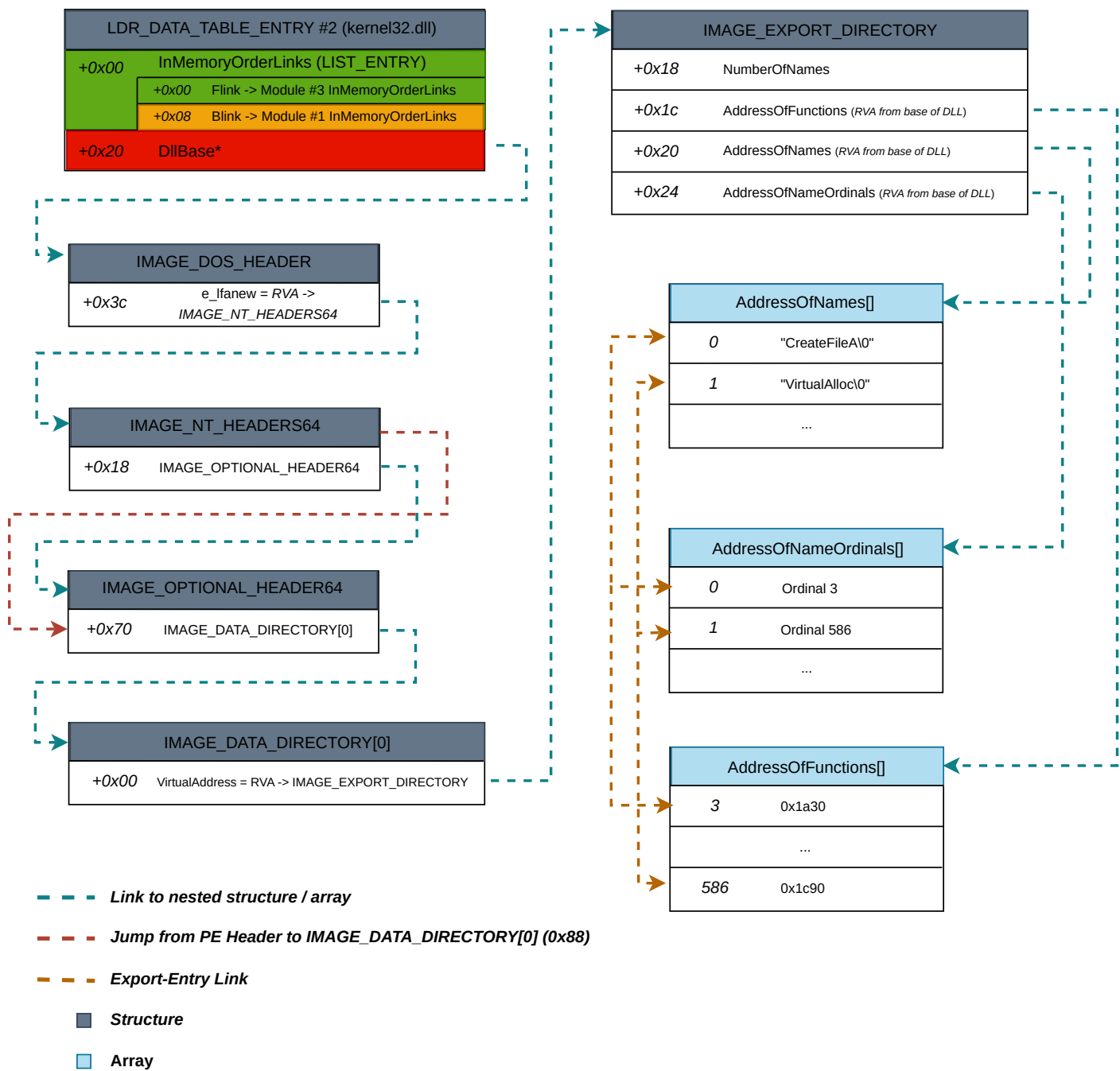
Alignment — rsp must be 16-byte aligned at the point of the call. Since call pushes 8 bytes, rsp is misaligned by 8 on entry, and the prologue has to correct for it.

```
sub  rsp, 28h          ; 20h shadow + 8 to realign
mov  rcx, 1             ; args 1-4 in registers
mov  rdx, 2
mov  r8, 3
mov  r9, 4
mov  qword ptr [rsp+20h], 5 ; arg 5 into its slot
call FiveArgFunction
add  rsp, 28h
```

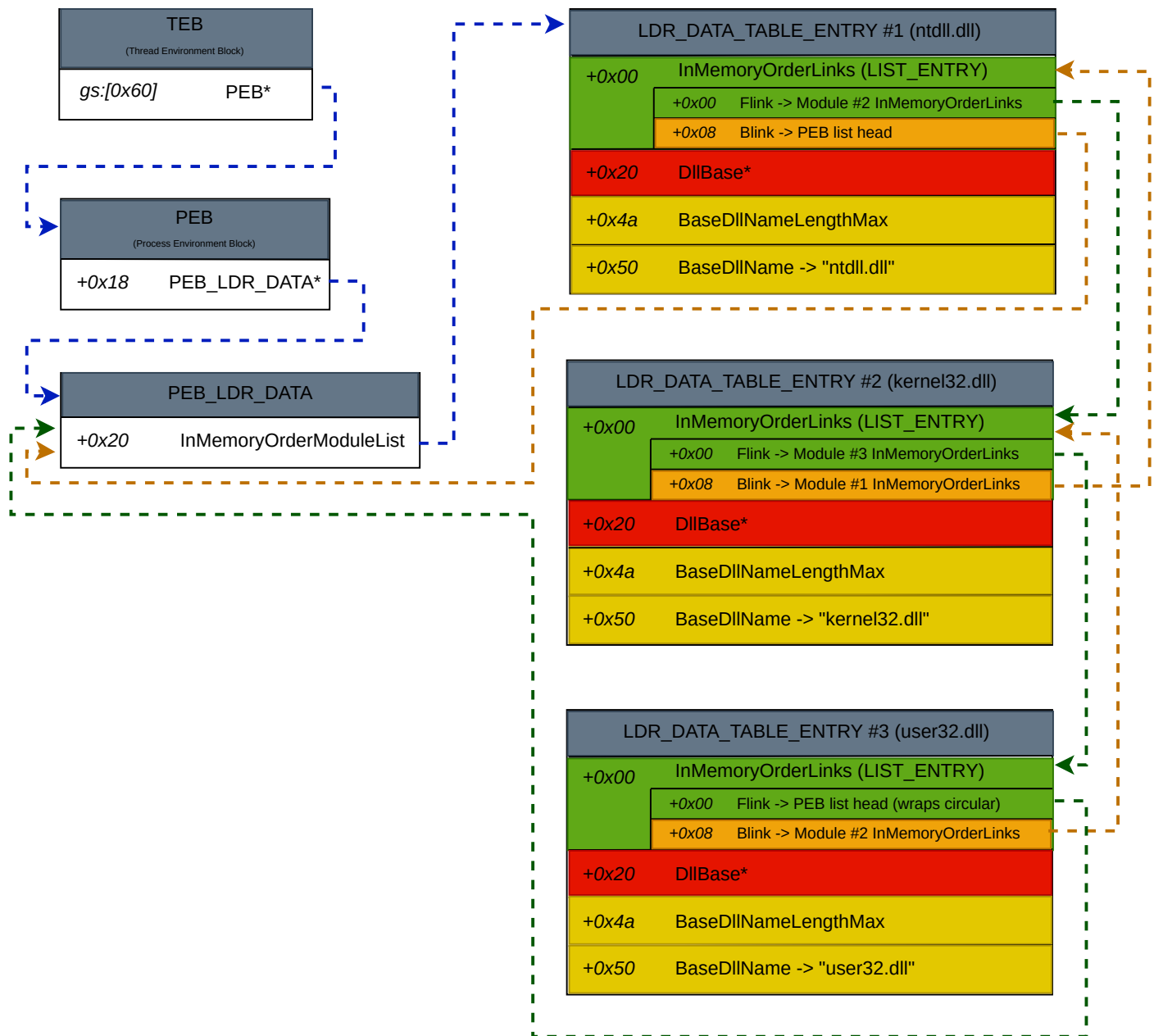
Arguments 5 and up are written into their slots with mov, not pushed. Allocate the frame once and leave rsp alone across the call.

Return values

- Integers, pointers, and booleans come back in rax.
- Anything larger than 8 bytes is returned through a **hidden pointer**: the caller allocates the space and passes its address as an invisible first argument, shifting every real argument one slot to the right.
- Values also come back through **out-parameters**, where the callee writes through a pointer you supplied. Win32 uses this heavily — consult the function's documentation to identify which arguments are outputs.



Portable Executable format, and where the export directory can be found in the PE.



Locating modules from within the PEB.

---Start of PROCESS_INFORMATION Struct---

0x00 (QWORD 1)

HANDLE hProcess (8 Bytes)

0x08 (QWORD 2)

HANDLE hThread (8 Bytes)

0x10 (QWORD 3)

DWORD dwProcessId (4 Bytes)

DWORD dwThreadId (4 Bytes)

---Start of STARTUPINFO Struct---

0x18 (QWORD 4)

DWORD cb (4 Bytes)

PADDING (4 Bytes)

0x20 (QWORD 5)

LPSTR lpReserved (8 Bytes)

0x28 (QWORD 6)

LPSTR lpDesktop (8 Bytes)

0x30 (QWORD 7)

LPSTR lpTitle (8 Bytes)

0x38 (QWORD 8)

DWORD dwX (4 Bytes)

DWORD dwY (4 Bytes)

0x40 (QWORD 9)

DWORD dwXSize (4 Bytes)

DWORD dwYSize (4 Bytes)

0x48 (QWORD 10)

DWORD dwXCountChars (4 Bytes)

DWORD dwYCountChars (4 Bytes)

0x50 (QWORD 11)

DWORD dwFillAttribute (4 Bytes)

DWORD dwFlags (4 Bytes)

0x58 (QWORD 12)

WORD wShowWindow (2 Bytes)

WORD cbReserved2 (2 Bytes)

PADDING (4 Bytes)

0x60 (QWORD 13)

LPBYTE lpReserved2 (8 Bytes)

---End of QWORD Loop---

0x68

HANDLE hStdInput (8 Bytes)

0x70

HANDLE hStdOutput (8 Bytes)

0x78

HANDLE hStdError (8 Bytes)

How the STARTUPINFO and PROCESS_INFORMATION structures look on the stack.